

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
VIZIANAGARAM
VIZIANAGARAM – 535003
ANDHRA PRADESH

FUEL CONSUMPTION TRACKER

By

PINNELLI AVINASH REDDY
(Regd. No.24VV1A1248)

Under the supervision of

MRS. MADHUMITA CHANDA

Department of Information Technology

Vizianagaram – 535003



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

Regd.No:25VV5A1272

This is to certify that this is a bonafide record of practical work done by P.KASI RAMA SUBRAMANYAM of III B.Tech I Semester Class in USER INTERFACEDESIGN USING FLUTTER Lab during the year 2025-26.

No.of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

Index

S.No	Contents	Page no	Marks	Signature
1	Introduction	1		
2	Project Structure	1		
3	Database Setup (SQLite)	3		
4	Flutter Installation and PATH Setup	4		
5	Dart SDK Installation	5		
6	Installing Visual Studio Code	6		
7	Creating a New Flutter Project	7		
8	Dart Programming Basics	7		
9	Case Study: Electricity Usage Tracker	9		
10	Exploring Flutter Widgets	10		
11	Responsive UI Design	11		
12	Navigation Between Screens	12		
13	Stateful and Stateless Widgets	13		
14	Custom Widgets and Styling	15		
15	Forms and Validation	16		
16	Animations	17		
17	Local Data Layer and State	18		
18	Testing and Debugging	20		
19	Workflow of Screens	20		
20	Conclusion	22		

1. Introduction

Flutter is a cross-platform toolkit from Google that builds native-looking interfaces from one Dart codebase. The widget model makes it possible to compose screens from small, reusable parts and to keep behaviour close to the interface that uses it.

The **Electricity Usage Tracker** is a Flutter application for recording domestic electricity meter readings. A user creates a local account, enters a previous and current reading, calculates the units consumed and the estimated bill, saves the result, and reviews the accumulated history through lists, charts, and alerts.

Project Aim

The aim is to provide a simple offline record of electricity use. Instead of keeping paper notes, the user has one place where each reading is stored with its date, tariff, units, and estimated cost.

Objectives

- Register and authenticate a local user.
- Capture previous and current meter values with validation.
- Calculate units from the difference between the two values.
- Estimate the bill by multiplying units by the tariff.
- Store readings and settings in a local SQLite database.
- Show history, summary metrics, charts, and usage alerts.

Technology Stack

Layer	Technology	Purpose
Interface	Flutter and Material 3	Screens and widgets
Language	Dart	Application logic and models
Storage	sqflite	Local SQLite databases
Charts	fl_chart	Usage bar and line charts
Preferences	shared_preferences	Saved theme mode
Platform	Android and desktop	Development targets

Report Organisation

The report begins with the project structure and local database, then covers the Flutter and Dart setup, the application case study, widgets, responsive design, navigation, state, forms, animation, testing, and the complete screen workflow. Code excerpts are taken from the project source and shortened to the relevant lines.

2. Project Structure

The project separates screens, models, database access, services, and theme configuration. This keeps a screen focused on presentation while shared data operations remain in the service layer.

Directory Layout

```
electricity_usage_tracker/
+-- lib/
|   +-- main.dart
|   +-- database/
|   |   +-- database_helper.dart
|   +-- models/
|   |   +-- electricity_reading.dart
|   +-- services/
|   |   +-- database_service.dart
|   +-- screens/
|   |   +-- splash_screen.dart
|   |   +-- landing_screen.dart
|   |   +-- login_screen.dart
|   |   +-- register_screen.dart
|   |   +-- dashboard_screen.dart
|   |   +-- add_reading_screen.dart
|   |   +-- bill_estimate_screen.dart
|   |   +-- usage_history_screen.dart
|   |   +-- analytics_screen.dart
|   |   +-- alerts_screen.dart
|   |   +-- profile_screen.dart
|   |   +-- personal_information_screen.dart
|   +-- theme/
|   |   +-- app_theme.dart
|   |   +-- theme_controller.dart
+-- assets/
+-- test/
+-- pubspec.yaml
```

Folder Responsibilities

Folder	Responsibility
lib/	Dart source and application entry point
database/	User table connection and creation
models/	Typed reading model and row mapping
services/	Reading, tariff, and delete operations
screens/	Page widgets and user interactions
theme/	Light and dark styles plus saved mode
assets/	Static resources declared in pubspec
test/	Widget and unit test sources

Application Entry Point

The root widget loads the saved theme before opening the splash screen. AnimatedBuilder rebuilds MaterialApp when the theme changes.

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ElectricityUsageTracker());
}

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: _themeController,
    builder: (context, child) {
      return MaterialApp(
        theme: AppTheme.lightTheme,
        darkTheme: AppTheme.darkTheme,
        themeMode: _themeController.themeMode,
        home: const SplashScreen(),
      );
    },
  );
}
```

Separation of Concerns

Screens do not construct SQL statements. They call a service or helper, receive typed data, and update local widget state. The theme controller is the only global notifier and is limited to appearance. This keeps the data layer replaceable and testable.

3. Database Setup (SQLite)

The application stores data on the device with sqflite. Two database files are currently present: electricity_tracker.db for user accounts and electricity_usage.db for readings and the tariff setting. Both are created on first use and require no remote server.

Users Table

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL
);
```

The unique email constraint prevents a duplicate account. Login queries the table by email and password. Passwords are currently stored as plain text, which is acceptable only for a local prototype and should be replaced with a hash before release.

Readings Table

```
CREATE TABLE readings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  previous_reading REAL NOT NULL,
  current_reading REAL NOT NULL,
  units REAL NOT NULL,
  bill_amount REAL NOT NULL,
  date TEXT NOT NULL
);
```

Settings Table

```
CREATE TABLE settings (
  id INTEGER PRIMARY KEY,
  tariff REAL NOT NULL
);
```

Database Versions

File	Version	Tables
electricity_tracker.db	1	users
electricity_usage.db	3	readings, settings

Opening the Database

```
Future<Database> get database async {
  if (_database != null) return _database!;
  final path = join(
    await getDatabasesPath(),
    'electricity_usage.db',
  );
  _database = await openDatabase(
    path,
    version: 3,
    onCreate: _createDatabase,
    onUpgrade: _upgradeDatabase,
  );
  return _database!;
}
```

Version 2 added the settings table and version 3 added the user_id column to readings. The duplicated user and reading layers should be consolidated into one database service in a future revision.

4. Flutter Installation and PATH Setup

Flutter is distributed as a compressed SDK. The installation does not need administrator rights if the SDK is extracted to a user-controlled folder. Android development also requires the Android command-line tools, platform tools, and a device or emulator.

Installation Steps

1. Download the stable Flutter SDK archive for Windows.
2. Extract the archive to a short path such as C:\src\flutter.
3. Add C:\src\flutter\bin to the user PATH variable.
4. Open a new terminal so the updated PATH is loaded.
5. Run flutter doctor and follow the reported fixes.
6. Connect a device or start an emulator before the first run.

PATH Configuration

```
C:\src\flutter\bin
C:\Users\<name>\AppData\Local\Android\Sdk\platform-tools
```

Verify the Installation

```
flutter --version
dart --version
flutter doctor
flutter devices
```

The version commands confirm the SDK, flutter doctor checks the toolchain, and flutter devices lists a target for deployment.

Insert screenshot showing:

Flutter doctor output after a successful installation

Figure 1: Flutter doctor output after a successful installation

Common Problems

Symptom	Likely Cause	Action
flutter is not recognised	PATH not refreshed	Open a new terminal
Android licence missing	SDK not accepted	Run licence command
No devices found	Emulator not started	Start an emulator
Build tools missing	Incomplete SDK	Install build tools

5. Dart SDK Installation

Flutter bundles the Dart SDK, so a separate Dart installation is not required for normal application work. A standalone SDK is useful only for running Dart scripts outside a Flutter project.

Check the Bundled Version

```
flutter --version
dart --version
where dart
```

Standalone Installation

1. Download the Dart SDK archive for Windows.
2. Extract it to a folder such as C:\dart-sdk.
3. Add C:\dart-sdk\bin to PATH.
4. Open a new terminal and run dart --version.

Environment Variables

```
DART_SDK = C:\dart-sdk
PATH = %PATH%;C:\dart-sdk\bin
```

Useful SDK Commands

Command	Use
dart --version	Show the active Dart version
dart run file.dart	Run a standalone script
dart pub get	Resolve package dependencies
dart format .	Format source files
dart analyze	Report static analysis issues

If two Dart versions are installed, the first matching entry in PATH is used. Place the Flutter bundled SDK first when working inside the application and keep the standalone SDK for scripts.

6. Installing Visual Studio Code

Visual Studio Code is a light editor with a strong Flutter workflow. It provides syntax highlighting, refactoring, an integrated terminal, and a debugger for Android and desktop.

Installation Steps

1. Download and install Visual Studio Code for Windows.
2. Open the Extensions view with Ctrl + Shift + X.
3. Install the Flutter extension.
4. Allow the extension to install the Dart dependency.
5. Open the project folder and wait for analysis to finish.

Required Extensions

Extension	Purpose
Flutter	Run, debug, and hot reload
Dart	Language analysis and editing
Flutter Widget Snippets	Reusable widget templates

Insert screenshot showing:

Flutter and Dart extensions installed in Visual Studio Code

Figure 2: Flutter and Dart extensions installed in Visual Studio Code

Useful Shortcuts

Shortcut	Action
Ctrl + Shift + P	Open the command palette
Ctrl + `	Open the integrated terminal
F5	Start debugging
Ctrl + Shift + F5	Restart the application
Shift + Alt + F	Format the current file

The status bar shows the active device and the Flutter version. If analysis does not start, confirm that the folder contains pubspec.yaml and run flutter pub get.

7. Creating a New Flutter Project

A new project is created from the Flutter command line. The project name uses lower-case words separated by underscores, which becomes the Dart package name.

Create and Run

```
flutter create electricity_usage_tracker
cd electricity_usage_tracker
flutter pub get
flutter run
```

pubspec.yaml

The application declares only the packages used by the current features. Versions are resolved by the package manager.

```
environment:
  sdk: ^3.12.2

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  sqflite: ^2.4.3
  path: ^1.9.1
  fl_chart: ^1.2.0
  shared_preferences: ^2.5.6
```

First Build Checks

- Confirm the application name in pubspec.yaml.
- Confirm the Android application ID in the Gradle files.
- Check that the assets folder is declared.
- Run flutter analyze before the first commit.

Insert screenshot showing:

New Flutter project opened with its folder structure

Figure 3: New Flutter project opened with its folder structure

Project Configuration

The display name appears in the Android manifest and the window title. The package identifier must remain unique if the project is later published. The assets folder is declared in pubspec so images and icons can be loaded at runtime.

8. Dart Programming Basics

Dart is strongly typed, null-safe, and class-based. The project uses simple variables, conditions, loops, model classes, and asynchronous methods for database access.

Variables and Types

```
String name = 'Electricity Tracker';
int readingCount = 4;
double tariff = 7.0;
bool isDarkMode = true;

var units = 125.5;      // inferred as double
final createdAt = DateTime.now();
const maxReadings = 500;
```

Conditions

```
double current = 1435;
double previous = 1250;

if (current < previous) {
  print('Current reading is too small');
} else if (current == previous) {
  print('No consumption recorded');
} else {
  print('Units: ${current - previous}');
}
```

Lists and Loops

```
final months = ['Jan', 'Feb', 'Mar'];

for (final month in months) {
  print(month);
}

for (var i = 0; i < months.length; i++) {
  print('${i + 1}. ${months[i]}');
}
```

Classes and Models

```
class ElectricityReading {
  final int? id;
  final int userId;
  final double units;
  final double billAmount;

  ElectricityReading({
    this.id,
    required this.userId,
    required this.units,
    required this.billAmount,
  });
}
```

Asynchronous Code

```
Future<List<ElectricityReading>> loadReadings(int userId) async {
  final rows = await DatabaseService.instance.getReadings(userId);
  return rows;
}

Future<void> openDashboard() async {
  final readings = await loadReadings(1);
  print('Loaded ${readings.length} readings');
}
```

Language Concepts Used

Concept	Where It Appears
Null safety	Optional id and nullable query results
Classes	ElectricityReading model
Futures	Database reads and writes
Collections	Reading lists and chart points
Named parameters	Widget and model constructors
Exceptions	Save and load failure handling

9. Case Study: Electricity Usage Tracker

The case study applies the language and Flutter material to a complete offline utility. A user can record readings, calculate usage, estimate a bill, and review the result without an internet connection.

Main Features

- Local registration, login, and logout.
- Profile and name editing.
- Add, list, and delete meter readings.
- Units and bill calculation with validation.
- Monthly usage summaries and charts.
- High-usage alerts and energy-saving tips.
- Light and dark theme selection.

User Journey

```
Launch application
      |
Splash and theme loading
      |
Landing screen
      |
Register or login
      |
Dashboard
      |
Add reading -> calculate -> save
      |
History, analytics, alerts, profile
```

Calculation Rule

Value	Formula	Example
Units	current - previous	1435 - 1250 = 185
Bill	units x tariff	185 x 7.0 = 1295.00
Average	total units / entries	Used in analytics

Insert screenshot showing:

Dashboard with summary cards and recent readings

Figure 4: Dashboard with summary cards and recent readings

Insert screenshot showing:

Bill estimate screen with a temporary calculation

Figure 5: Bill estimate screen with a temporary calculation

Design Goals

- Keep the main actions reachable from the dashboard.
- Explain calculations before a reading is saved.
- Use clear validation messages instead of silent failure.
- Remain readable in both light and dark appearance.

10. Exploring Flutter Widgets

Every visible part of a Flutter screen is a widget. Layout widgets position children, material widgets provide interaction and styling, and custom widgets combine both into reusable blocks.

Widgets Used in the Project

Widget	Role
MaterialApp	Root theme and route configuration
Scaffold	Page structure and app bar
Column / Row	Vertical and horizontal layout
Container	Padding, border, and decoration
Card	Grouped reading information
TextField	Meter and tariff input
ListView	Scrollable history
NavigationBar	Primary section switching

Dashboard Layout

```
Scaffold(
 appBar: AppBar(title: const Text('Dashboard')),
 body: ListView(
  padding: const EdgeInsets.all(16),
  children: [
   SummaryCards(readings: readings),
   const SizedBox(height: 12),
   SectionTitle('Recent Readings'),
   ReadingList(readings: readings),
  ],
 ),
)
```

Widget Lifecycle

- initState loads data and creates controllers.
- build describes the interface for the current state.
- didUpdateWidget reacts to changed parent values.
- dispose releases controllers and listeners.

Insert screenshot showing:

Reading list with saved meter entries

Figure 6: Reading list with saved meter entries

Choosing a Widget Type

Type	Use When
StatelessWidget	The output depends only on constructor data
StatefulWidget	The screen owns changing local state

11. Responsive UI Design

The interface must remain usable on a phone, a small tablet, and a resizable desktop window. Responsive design avoids hard-coded widths and lets content reflow when space changes.

Techniques

- Use Expanded and Flexible instead of fixed widths.
- Wrap long pages in a scrollable view.
- Switch from a column to a row when width allows.
- Constrain content width on large screens.
- Test with a narrow window and a large text scale.

LayoutBuilder Example

```
LayoutBuilder(
  builder: (context, constraints) {
    final isWide = constraints.maxWidth >= 700;
    final cards = [
      SummaryCard(title: 'Units', value: '185'),
      SummaryCard(title: 'Bill', value: '1295'),
    ];
    return isWide
        ? Row(children: cards)
        : Column(children: cards);
  },
)
```

Suggested Breakpoints

Width	Layout
Below 600 pt	Single column and compact spacing
600 to 900 pt	Two-column cards where possible
Above 900 pt	Constrained content and wider charts

Insert screenshot showing:

Dashboard adapting to a narrow mobile screen

Figure 7: Dashboard adapting to a narrow mobile screen

Layout Checks

- No horizontal scrolling at the smallest supported width.
- Buttons remain reachable with one hand.
- Chart labels do not overlap.
- Long validation messages wrap instead of clipping.

12. Navigation Between Screens

Navigation moves the user from the public entry pages to the authenticated dashboard and then into individual features. The project uses direct MaterialPageRoute pushes and a bottom navigation bar for primary sections.

Screen Flow

```
Splash -> Landing -> Login -> Dashboard
                    |
                 Register

Dashboard -> Add Reading
Dashboard -> Bill Estimate
Dashboard -> Usage History
Dashboard -> Analytics
Dashboard -> Alerts
Dashboard -> Profile
```

Push and Pop

```
Navigator.push(
 context,
 MaterialPageRoute(
  builder: (context) => AddReadingScreen(
   userId: user.id,
  ),
 ),
);

Navigator.pop(context);
```

Bottom Navigation

```
NavigationBar(
 selectedIndex: index,
 onDestinationSelected: (value) {
  setState(() => index = value);
 },
 destinations: const [
  NavigationDestination(
   icon: Icon(Icons.dashboard),
   label: 'Home',
  ),
  NavigationDestination(
   icon: Icon(Icons.history),
   label: 'History',
  ),
 ],
)
```

Navigation Rules

- Pass the authenticated user id to feature screens.
- Clear the back stack after logout.
- Return a result only when the caller needs a refresh signal.
- Keep route names consistent with screen class names.

Insert screenshot showing:

Login screen before entering the dashboard

Figure 8: Login screen before entering the dashboard

13. Stateful and Stateless Widgets

Stateless widgets describe a fixed view from their inputs. Stateful widgets own values that change while the screen is open, such as form text, loading flags, and calculated totals.

Stateless Example

```
class SummaryCard extends StatelessWidget {
 final String label;
 final String value;
 final IconData icon;

 const SummaryCard({
  super.key,
  required this.label,
  required this.value,
  required this.icon,
 });

 @override
 Widget build(BuildContext context) {
  return Card(
   child: ListTile(
    leading: Icon(icon),
    title: Text(label),
    subtitle: Text(value),
   ),
  );
 }
}
```

Stateful Example

```
class AddReadingScreen extends StatefulWidget {
 final int userId;
 const AddReadingScreen({super.key, required this.userId});

 @override
 State<AddReadingScreen> createState() =>
    _AddReadingScreenState();
}

class _AddReadingScreenState extends State<AddReadingScreen> {
 final previousController = TextEditingController();
 final currentController = TextEditingController();
 double units = 0;
 bool isSaving = false;
}
```

Lifecycle Methods

Method	Purpose
initState	Create controllers and load initial data
build	Describe the current frame
setState	Request a rebuild after a state change
dispose	Release controllers and listeners

State Rules

- Keep state as close as possible to the widget that uses it.
- Do not call setState after the widget is disposed.

- Use a service or notifier for state shared by many screens.
- Avoid storing derived values when they can be computed.

14. Custom Widgets and Styling

Custom widgets remove repeated layout code and keep visual rules in one place. The project defines theme colours for light and dark mode and reuses card, button, and section patterns across the screens.

Theme Definition

```
ThemeData get lightTheme => ThemeData(
 brightness: Brightness.light,
 colorScheme: ColorScheme.fromSeed(
  seedColor: const Color(0xFF1769D2),
 ),
 useMaterial3: true,
);

ThemeData get darkTheme => ThemeData(
 brightness: Brightness.dark,
 colorScheme: ColorScheme.fromSeed(
  seedColor: const Color(0xFF00B7FF),
  brightness: Brightness.dark,
 ),
 useMaterial3: true,
);
```

Reusable Card

```
class InfoCard extends StatelessWidget {
 final String title;
 final String value;
 final IconData icon;

 const InfoCard({
  super.key,
  required this.title,
  required this.value,
  required this.icon,
 });

 @override
 Widget build(BuildContext context) {
  return Card(
   child: ListTile(
    leading: Icon(icon),
    title: Text(title),
    subtitle: Text(value),
   ),
  );
 }
}
```

Theme Controller

```
Future<void> setTheme(ThemeMode mode) async {
 final preferences = await SharedPreferences.getInstance();
 await preferences.setString(
  'app_theme_mode',
  mode == ThemeMode.light ? 'light' : 'dark',
 );
 _themeMode = mode;
 notifyListeners();
}
```

Insert screenshot showing:

Profile screen with theme selection controls

Figure 9: Profile screen with theme selection controls

Styling Rules

- Read colour and text styles from Theme.of(context).
- Use one spacing scale for padding and gaps.
- Keep card elevation low and borders subtle.
- Check contrast in both brightness modes.

15. Forms and Validation

The reading form is the main data entry point. It accepts three numeric values, validates them before calculation, shows the computed units and bill, and saves the result only after a successful check.

Reading Form Fields

Field	Input	Validation
Previous reading	Decimal number	Required, not negative
Current reading	Decimal number	Required, not below previous
Tariff	Decimal number	Required, positive
Date	System timestamp	Generated on save

Validation Rules

Condition	Response
Empty field	Show a required-field message
Non-numeric input	Reject without calculation
Negative value	Reject as invalid
Current below previous	Reject as inconsistent

Validation Logic

```
({double previous, double current, double rate})? readInputs() {
 final previous = double.tryParse(previousController.text);
 final current = double.tryParse(currentController.text);
 final rate = double.tryParse(rateController.text);

 if (previous == null || current == null || rate == null) {
  showMessage('Enter valid numbers', error: true);
  return null;
 }
 if (previous < 0 || current < 0 || rate < 0) {
  showMessage('Values cannot be negative', error: true);
  return null;
 }
 if (current < previous) {
  showMessage('Current reading is too small', error: true);
  return null;
 }
 return (previous: previous, current: current, rate: rate);
}
```

Error Feedback

- Use a snackbar for a short, visible message.
- Keep the invalid field focused for correction.
- Do not save or calculate after a failed check.

Insert screenshot showing:

Add reading form with units and bill result

Figure 10: Add reading form with units and bill result

16. Animations

Animation is used sparingly to make state changes easier to follow. Short fades and size changes are enough for cards, loading states, and result panels.

Implicit Animation

```
AnimatedContainer(
 duration: const Duration(milliseconds: 250),
 curve: Curves.easeOut,
 padding: EdgeInsets.all(isSelected ? 18 : 12),
 decoration: BoxDecoration(
  borderRadius: BorderRadius.circular(18),
  border: Border.all(
   color: isSelected ? Colors.blue : Colors.grey,
  ),
 ),
 child: const Text('Reading summary'),
)
```

Animated Switcher

```
AnimatedSwitcher(
 duration: const Duration(milliseconds: 300),
 child: isSaving
    ? const CircularProgressIndicator()
    : const Icon(Icons.check_circle),
)
```

Custom Painter

The add-reading screen uses a CustomPainter for a decorative energy background. The painter draws soft circles, a grid, and a moving wave. The animation repeats slowly and is shown only in dark mode so that it does not distract from the form.

Guidelines

- Keep durations between 200 and 400 milliseconds.
- Animate one property group at a time.
- Respect reduced-motion expectations on long animations.
- Dispose animation controllers when the screen closes.

Pattern	Best Use
AnimatedContainer	Size, colour, and padding changes
AnimatedSwitcher	Replacing a child with a new state
AnimationController	Repeating or custom motion

17. Local Data Layer and State

The data layer hides SQL details behind service methods. A screen calls insertReading or getReadings and receives a model or a list. This keeps persistence out of the widget tree.

Insert a Reading

```
Future<int> insertReading(
 ElectricityReading reading,
) async {
 final db = await database;
 return await db.insert('readings', reading.toMap());
}
```

Query Readings

```
Future<List<ElectricityReading>> getReadings(
 int userId,
) async {
 final db = await database;
 final result = await db.query(
  'readings',
  where: 'user_id = ?',
  whereArgs: [userId],
  orderBy: 'id DESC',
 );
 return result.map(ElectricityReading.fromMap).toList();
}
```

Tariff Setting

```
Future<double> getTariff() async {
 final db = await database;
 final result = await db.query(
  'settings',
  where: 'id = ?',
  whereArgs: [1],
  limit: 1,
 );
 if (result.isEmpty) return 7.0;
 return (result.first['tariff'] as num).toDouble();
}
```

Theme State

```
class ThemeController extends ChangeNotifier {
 static final ThemeController instance =
    ThemeController._();
 ThemeController._();

 ThemeMode _themeMode = ThemeMode.dark;
 ThemeMode get themeMode => _themeMode;

 Future<void> loadTheme() async {
  final preferences =
     await SharedPreferences.getInstance();
  _themeMode = preferences.getString('app_theme_mode') == 'light'
     ? ThemeMode.light
     : ThemeMode.dark;
  notifyListeners();
 }
}
```

State Boundaries

State	Owner
Form text and validation	AddReadingScreen
Saved readings	DatabaseService and screen cache
Tariff	Settings table
Appearance mode	ThemeController

Insert screenshot showing:

Analytics screen with summary cards and charts

Figure 11: Analytics screen with summary cards and charts

Error Handling

```
try {
  await DatabaseService.instance.insertReading(reading);
  showMessage('Reading saved');
} catch (error) {
  showMessage('Could not save reading', error: true);
} finally {
  if (mounted) setState(() => isSaving = false);
}
```

18. Testing and Debugging

Testing checks calculation rules, form validation, database operations, and the main screen flow. Debugging tools help inspect widgets, layout, and asynchronous work.

Widget Test

```
testWidgets('shows the login action', (tester) async {
  await tester.pumpWidget(
    const MaterialApp(home: LoginScreen()),
  );
  expect(find.text('Login'), findsWidgets);
  expect(find.byType(TextFormField), findsNWidgets(2));
});
```

Manual Test Cases

Case	Expected Result
Register with a new email	Account is created
Register with an existing email	Duplicate message appears
Login with wrong password	Access is denied
Enter current below previous	Reading is rejected
Save a valid reading	Reading appears in history
Delete a reading	Entry is removed from the list

Debugging Tools

- Use the VS Code debugger and breakpoints.
- Open the Flutter inspector for layout problems.
- Run flutter analyze before a build.
- Use flutter logs to view device output.

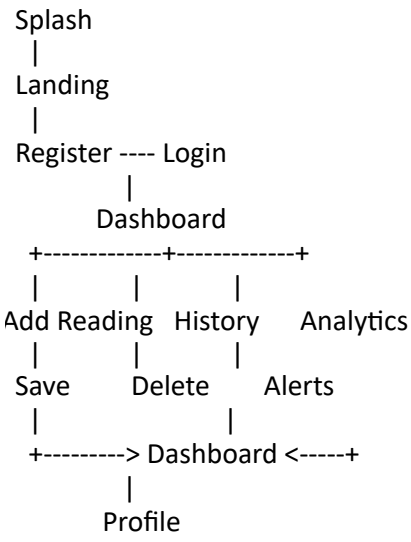
Current Test Status

The repository still contains the default counter widget test. It does not exercise the application screens and should be replaced with tests for authentication, validation, calculation, and reading persistence.

19. Workflow of Screens

The screen workflow connects account access, data entry, review, and settings. The dashboard is the central hub; every feature returns to it after completion.

End-to-End Flow



Screen Responsibilities

Screen	Responsibility
Splash	Load the saved theme and show startup state
Landing	Introduce the app and open authentication
Register	Create a local account with validation
Login	Verify email and password
Dashboard	Show summary and recent readings
Add Reading	Validate, calculate, and save
History	List and delete saved readings
Analytics	Show averages, totals, and charts
Alerts	Display high-usage warnings and tips
Profile	Edit details and choose a theme

Screens to Capture

The placeholders in this report mark the exact screens that should be replaced with current application screenshots. The final checklist in the conclusion repeats the list.

Insert screenshot showing:

Registration screen with validation messages

Figure 12: Registration screen with validation messages

Insert screenshot showing:

Usage history listing saved readings

Figure 13: Usage history listing saved readings

Insert screenshot showing:

Alerts and energy-saving guidance

Figure 14: Alerts and energy-saving guidance

Failure Paths

- A failed login returns to the form with a message.
- An invalid reading is not saved.
- A database error leaves the form open for retry.
- Logout clears the active user and returns to landing.

20. Conclusion

The Electricity Usage Tracker demonstrates a complete local Flutter workflow: account handling, validated data entry, calculation, persistence, history, analytics, alerts, and theme selection. The interface remains responsive and the data layer is separated from the screens.

Achievements

- A working local authentication flow.
- Validated reading entry with units and bill calculation.
- SQLite persistence for readings and tariff settings.
- History, summary metrics, and chart-based review.
- Light and dark themes with a saved preference.

Limitations

- Passwords are stored in plain text.
- Two database layers overlap in responsibility.
- Automated test coverage is still limited.
- The default widget test is not application-specific.

Future Improvements

- Consolidate the user and reading databases.
- Hash passwords with a salted algorithm.
- Add application-specific unit and widget tests.
- Support per-user tariffs and export.
- Add a monthly budget and usage comparison.

Screenshot Checklist

Figure	Screen to Insert
1	Flutter doctor terminal output
2	VS Code with Flutter and Dart extensions
3	New Flutter project folder structure
4	Dashboard with summary cards
5	Bill estimate screen
6	Reading list with saved entries
7	Responsive dashboard on a narrow screen
8	Login screen
9	Profile and theme selection
10	Add reading form with result
11	Analytics screen with charts
12	Registration screen
13	Usage history screen
14	Alerts and energy-saving guidance

Replace each placeholder with a screenshot that matches its caption. Keep the phone frame, status bar, and visible labels consistent so the report reads as one application.